

AFRILANG Stdlib

std/c/parsehex

Bibliothèque AFRILANG `std/c/parsehex` (niveau complex, catégorie complex). Elle expose 6 fonction(s) : decode, encode,

```
`import "std/c/parsehex" . `use parsehex`
```

- `decode(hex text) ? list number`
- `encode(bytes list number) ? text`
- `nibbleCount(hex text) ? number`
- `byteCount(hex text) ? number`
- `isHex(s text) ? bool`
- `roundTrip(bytes list number) ? bool`

Category: complex | Tier: complex | Functions: 6

FRANCAIS

1. Vue d'ensemble

Bibliothèque AFRILANG `std/c/parsehex` (niveau complex, catégorie complex). Elle expose 6 fonction(s) : decode, encode,

```
`import "std/c/parsehex"` . `use parsehex`  
- `decode(hex text) ? list number`  
- `encode(bytes list number) ? text`  
- `nibbleCount(hex text) ? number`  
- `byteCount(hex text) ? number`  
- `isHex(s text) ? bool`  
- `roundTrip(bytes list number) ? bool`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/parsehex"  
use parsehex
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés ? graphes, NLP, réseaux complexes.

3. Référence API

```
? decode(hex text) ? list  
? encode(bytes list number) ? text  
? nibbleCount(hex text) ? number  
? byteCount(hex text) ? number  
? isHex(s text) ? bool  
? roundTrip(bytes list number) ? bool
```

4. Exemples d'utilisation

```
import "std/c/parsehex"  
use parsehex  
  
create result = decode(/* args */)   
say result  
  
create result = encode(/* args */)   
say result  
  
create result = nibbleCount(/* args */)   
say result  
  
create result = byteCount(/* args */)   
say result  
  
create result = isHex(/* args */)   
say result  
  
create result = roundTrip(/* args */)   
say result
```

5. Bonnes pratiques

? Préférez `import "std/c/parsehex"` en tête de fichier.
? Lancez `afrilang check` avant de livrer.
? Combinez avec les modules stdlib de la même catégorie.
? Voir /stdlib/ pour le source live et les démos playground.
? Signalez les problèmes sur GitHub si une export manque.

6. Annexe ? source

```
module parsehex  
  
function decode(hex text) returns list number  
end
```

```
function encode(bytes list number) returns text
end

function nibbleCount(hex text) returns number
end

function byteCount(hex text) returns number
end

function isHex(s text) returns bool
end

function roundTrip(bytes list number) returns bool
end
end
```

ENGLISH

1. Overview

AFRILANG library `std/c/parsehex` (tier complex, category complex). Exports 6 function(s): decode, encode, nibbleCount,

```
`import "std/c/parsehex"` . `use parsehex`  
- `decode(hex text) ? list number`  
- `encode(bytes list number) ? text`  
- `nibbleCount(hex text) ? number`  
- `byteCount(hex text) ? number`  
- `isHex(s text) ? bool`  
- `roundTrip(bytes list number) ? bool`
```

2. Installation & import

Add the module to your program:

```
import "std/c/parsehex"  
use parsehex
```

Tier: complex · Category: Complex modules (c/)
Advanced domains ? graphs, NLP, complex networks.

3. API reference

```
? decode(hex text) ? list  
? encode(bytes list number) ? text  
? nibbleCount(hex text) ? number  
? byteCount(hex text) ? number  
? isHex(s text) ? bool  
? roundTrip(bytes list number) ? bool
```

4. Usage examples

```
import "std/c/parsehex"  
use parsehex  
  
create result = decode(/* args */)   
say result  
  
create result = encode(/* args */)   
say result  
  
create result = nibbleCount(/* args */)   
say result  
  
create result = byteCount(/* args */)   
say result  
  
create result = isHex(/* args */)   
say result  
  
create result = roundTrip(/* args */)   
say result
```

5. Best practices

```
? Prefer `import "std/c/parsehex"` at the top of your file.  
? Run `afrilang check` before shipping.  
? Combine with related stdlib modules from the same category.  
? See /stdlib/ for the live source and playground demos.  
? Report issues on GitHub if an export is missing or undocumented.
```

6. Appendix ? source

```
module parsehex  
  
function decode(hex text) returns list number  
end
```

```
function encode(bytes list number) returns text
end

function nibbleCount(hex text) returns number
end

function byteCount(hex text) returns number
end

function isHex(s text) returns bool
end

function roundTrip(bytes list number) returns bool
end
end
```